

Dioris Planner V2 — Step 8.2

Generic Carcass Semantics + Acceptance Completeness Lock

Estado: concluída tecnicamente. **Stage 9:** não iniciada.

1. Baseline de entrada

Antes da correção, a suíte tinha **51 arquivos e 585 testes aprovados**. O TypeScript apresentava exatamente cinco erros preexistentes em `usePlannerStore.ts`, linhas 1089–1093. O production build real terminou com exit code 0.

O teste dirigido pré-fix reproduziu o blocker: com `kitchen-golden-upper-800`, dimensões `800 × 700 × 350`, `toeKickMm = 0` e três prateleiras, o resolver retornava `INVALID_DIMENSIONS`, criava uma representação `toeKick` de altura zero e fazia as relações das laterais referenciarem `toe-kick`.

2. Causa raiz

O wiring da Step 8.1 estava correto: a definição Upper chegava à factory, `moduleDefinitionId` era preservado e os três resolvers eram chamados. O problema restante estava no contrato Carcass, originalmente desenhado para o balcão inferior.

Esse contrato exigia `sideRelation = full-height-above-toe-kick`, exigia `toeKickRelation = separate-profile-supported-by-feet` e sempre retornava um objeto `toeKick`. A validação também tratava `toeKickMm ≤ 0` como dimensão inválida. Assim, o Upper era semanticamente correto como móvel, mas inválido no resolved carcass.

3. Contrato genérico corrigido

O contrato agora distingue explicitamente:

Semântica	Base	Upper
<code>sideRelation</code>	<code>full-height-above-toe-kick</code>	<code>full-height</code>
<code>toeKickRelation</code>	<code>separate-profile-supported-by-feet</code>	<code>none</code>
<code>ResolvedCarcass.toeKick</code>	presente	ausente (<code>undefined</code>)

Foram adicionados os diagnósticos `INVALID_TOE_KICK` , `UNEXPECTED_TOE_KICK` e `MISSING_REQUIRED_TOE_KICK` . Não foi introduzida uma exceção baseada em `moduleDefinitionId` .

4. Algoritmo de validação

Dimensões externas são validadas separadamente e devem ser finitas e positivas. O toe-kick deve ser finito e não negativo. Quando a regra é `none` , o valor exigido é exatamente zero. Quando a regra é `separate-profile-supported-by-feet` , o valor deve ser positivo.

A altura do corpo é derivada semanticamente. Para `none` , `bodyBottomMm = 0` e `bodyHeightMm = cabinetHeight` . Para a regra com rodapé, `bodyBottomMm = toeKickMm` e `bodyHeightMm = cabinetHeight - toeKickMm` . No Upper, isso mantém `bodyHeightMm = 700` e `internalHeightMm = 664` .

5. Resultado Upper READY

O Upper agora resolve como `READY` , sem diagnostics, com `toeKickMm = 0` e `toeKick` `≡ undefined` . As laterais são `full-height` , têm altura 700 mm, centro Y 350 mm e não referenciam `toe-kick` . A geometria das dez partes físicas e dos 22 itens de hardware não foi alterada.

6. Resultado Base READY

O Golden Base continua resolvendo como `READY` , com `toeKick` presente, altura positiva de 150 mm, relação `separate-profile-supported-by-feet` e laterais `full-height-above-toe-kick` . As dimensões, posições, prateleiras e downstreams anteriores permaneceram verdes.

7. Proteção contra INVALID silencioso

`buildCarcass()` agora rejeita uma resolução selecionada com status `INVALID` usando o mecanismo existente de erro do pipeline. `buildModule()` captura a falha e retorna o resultado não fabricável com diagnóstico. O teste fixture comprova que um Upper forçado com toe-kick positivo não é materializado silenciosamente.

8. Base + Upper na mesma store

O teste de wiring cria Base e Upper sem reset entre eles. As duas ocorrências têm IDs diferentes, definições corretas, resolvers `READY`, toe-kick presente apenas no Base, PartDefinitions instance-scoped e nenhum ID físico compartilhado.

9. Front application type

A auditoria mostrou que `FrontLayoutRule.applicationType` não possui consumidores operacionais. Foi adicionado o tipo declarativo `paired-overlay`. O Upper agora usa `paired-overlay` com `symmetric = false`; o Base continua usando `symmetric-paired-overlay` com `symmetric = true`. A assimetria operacional continua sendo controlada pelo campo `symmetric` e pelos reveals declarados.

10. Machining readiness

O acceptance test percorre as operações reais de Machining e verifica `id`, `partId`, `instanceId`, relações e readiness. Uma operação `READY` não pode ter parâmetros obrigatórios ausentes. Uma operação `INCOMPLETE`, quando existente, precisa carregar `missingParameters`; nenhum estado industrial foi inventado.

11. $A \rightarrow B \rightarrow A$ e multi-instance

O ciclo `800 → 850 → 800` preserva a instância e retorna cut-list, nesting e coordenadas locais determinísticas ao snapshot original. Duas instâncias Upper possuem IDs distintos, PartDefinition IDs distintos, group IDs isolados e operações de Joinery/Machining isoladas. Mover ou rotacionar no mundo não altera o snapshot local de fabricação.

12. Mutation checks

Mutação	Resultado
Forçar <code>toeKickMm ≤ 0</code> como inválido	Falhou como esperado, exit code 1
Trocar Upper para <code>separate-profile-supported-by-feet</code>	Falhou como esperado, exit code 1
Remover a proteção contra consumo de <code>ResolvedCarcass.INVALID</code>	Falhou como esperado, exit code 1

Cada mutação foi temporária, teve log completo e foi restaurada antes da validação final.

13. Validação final

Após a Step 8.2, a suíte terminou com **52 arquivos e 588 testes aprovados**, incluindo os novos locks. O TypeScript terminou com exatamente os cinco erros preexistentes em `usePlannerStore.ts`; erros novos da Step 8.2 foram eliminados. O production build terminou com exit code 0.

14. Arquivos e escopo

A evidência separa os arquivos sujos anteriores da Step 8.2 dos arquivos tocados nesta missão. A entrega inclui somente os contratos Carcass/Front relevantes, resolver, regras, builder, testes semânticos e acceptance tests da Step 8.2, além das evidências e relatórios.

Dívidas técnicas não tratadas: os cinco erros legados do store permanecem fora do escopo; os warnings de depreciação de `inputValidator()` permanecem informativos; CAM, CNC, G-code, nova família, gaveteiro, torre e Stage 9 continuam bloqueados.

Conclusão

A Step 8.2 fecha o blocker semântico: Base e Upper usam o mesmo engine, mas cada regra declara corretamente sua relação estrutural. O Base possui toe-kick real; o Upper não possui toe-kick nem como peça de altura zero. Ambos resolvem `READY`, e uma resolução `INVALID` não pode mais virar fabricação aparentemente válida.